UNIVERSITY OF BARISHAL

ICPC
CODEBOOK

TEAM NAME

BU_Out_Of_Limit

Competitive Programming Reference

GRAPH :

1.Articulation Point

```
int timer=0;
 void dfs(int node ,int parent,vector<int>adj[],vector<int>&vis,int tin[],int low[],vector<int>&mark){
    vis[node]=1;
tin[node]=low[node]=timer;
    timer++;
    int c=0;
    for(auto u:adj[node]){
    if(u==parent)continue;
      if(!vis[u]){
        c++;   dfs(u,node,adj,vis,tin,low,mark);
low[node]=min(low[node],low[u]);
if(low[u]>=tin[node]&&parent!=-1){
            mark[node]=1;
          }
        }
        else low[node]=min(low[node],tin[u]);
    }
    if(c>1&&parent==-1)mark[node]=1;
  }
 vector<int> articulationPoints(int V, vector<int>adj[]) {
    // Code here
    vector<int>vis(V,0);
    int tin[V];
    int low[V];
    vector<int>mark(V,0);
    for(int i=0;i<V;i++){
      if(!vis[i]){
        dfs(i,-1,adj,vis,tin,low,mark);
      }
    }
    vector<int>ans;
    for(int i=0;i<V;i++){
      if(mark[i])ans.push_back(i);
    }
    if(ans.size()==0)return {-1};
    return ans;
  }
```

2.BALLMANFORD

```
vector<int> bellman_ford(int V, vector<vector<int>>& edges, int S) {
    // Code here
    vector<int>dis(V,1e8);
    dis[S]=0;
    for(int i=0;i<V-1;i++){
      for(auto t:edges){
        int u=t[0];
        int v=t[1];
        int w=t[2];
        if(dis[u]==1e8)continue;
        if(dis[u]+w<dis[v]){
          dis[v]=dis[u]+w;
```

```
            }
          }
        }
        for(auto t:edges){
            int u=t[0];
            int v=t[1];
            int w=t[2];
            if(dis[u]==1e8)continue;
            if(dis[u]+w<dis[v]){
              return {-1};
            }
          }
          return dis;
    }
```

3. Bridges in a graph (Tarjans algorithm)

```
int time=1;
   void dfs(int node,vector<int>adj[],int tin[],int low[],int parent,vector<int>&vis,vector<vector<int>>&bridge){
      vis[node]=1;
      tin[node]=time;
      low[node]=time;
      time++;
      for(auto u:adj[node]){
        if(u==parent)continue;
        if(vis[u]==0){
           dfs(u,adj,tin,low,node,vis,bridge);
           low[node]=min(low[node],low[u]);
           if(low[u]>tin[node]){
            bridge.push_back({node,u});
           }
         }
         else low[node]=min(low[node],tin[u]);
      }
    }
   vector<vector<int>> criticalConnections(int n, vector<vector<int>>& connections) {
        vector<int>adj[n];
        for(auto u:connections){
           adj[u[0]].push_back(u[1]);
           adj[u[1]].push_back(u[0]);
        }
        int tin[n];
        int low[n];
        vector<int>vis(n,0);
        vector<vector<int>>bridge;
       for(int i=0;i<n;i++) if(!vis[i]) dfs(i,adj,tin,low,-1,vis,bridge);
       return bridge;
    }
```

4.Maximum bipartite matching

```
void answer();
const ll mx=1e5+123;
const ll N=2e5+123;
int dx[]={0,0,1,-1};
```

```
int dy[]={1,-1,0,0};
vector<vector<int>>adj(mx);
int vis[mx];
int ass[mx];
bool check(int u){
  for(auto v:adj[u]){
     if(vis[v]==0){
      vis[v]=1;
      if(ass[v]==-1||check(ass[v]))
         {
      ass[v]=u;
      return true;
         }
     }
  }
  return false;
}
int main()
{
   optimize();
    int t=1;
  // cin>>t;
   int tt=t;
    while(t--){
      // cout<<"Case "<<tt-t<<": ";
      answer();
    }
    return 0;
}
void answer()
{
 ll n;
 cin>>n;
 vector<ll>v(n);
 for(int i=0;i<n;i++)cin>>v[i];
 for(int i=0;i<n;i++){
   ll x=v[i];
   if(x<=n)adj[i+1].pb(x);
   vector<ll>bin;
   while(x>0){
      bin.pb(x%2);
      x/=2;
   }
   for(int j=0;j<bin.size();j++){
      if(bin[j]==1){
         ll y=v[i]-pow(2,j);
         if(y<=n&&y>0)adj[i+1].pb(y);
      }
   }
 }
 /*for(int i=1;i<=n;i++){
   cout<<i<<endl;
   for(auto u:adj[i])cout<<u<<" ";
   cout<<endl;
 }*/
 ll ans=0;
 memset(ass,-1,sizeof(ass));
```

```
 for(int i=1;i<=n;i++){
      memset(vis,0,sizeof(vis));
   if(check(i))ans++;
 }
 if(ans==n)cout<<"Yes"<<endl;
 else cout<<"No"<<endl;
}
```

5. Dijkstra algorithm

```
vector <int> dijkstra(int V, vector<vector<int>> adj[], int S)
   {
      // Code here
      set<pair<int,int>>q;
      vector<int>dis(V);
      for(int i=0;i<V;i++)dis[i]=1e9;
      dis[S]=0;
      q.insert({0,S});
      while(!q.empty()){
         auto it=*(q.begin());
         int d=it.first;
         int n=it.second;
         q.erase(it);
         for(auto u:adj[n]){
            if(d+u[1]<dis[u[0]]){
if(dis[u[0]]!=1e9)q.erase({dis[u[0]],u[0]});
               dis[u[0]]=d+u[1];
               q.insert({dis[u[0]],u[0]});
            }
         }
      }
      return dis;
   }
```

6. Floyd warshall algorithm

```
        void shortest_distance(vector<vector<int>>&matrix)
{
            // Code here
            int n=matrix.size();
            for(int i=0;i<n;i++){
               for(int j=0;j<n;j++){
                 if( matrix[i][j]==-1)matrix[i][j]=1e9;
               }
            }
            for(int i=0;i<matrix.size();i++){
               for(int j=0;j<matrix.size();j++){
                  for(int k=0;k<matrix[0].size();k++){
        if(matrix[j][i]!=1e9 && matrix[i][k]!=1e9) matrix[j][k]=min(matrix[j][k],matrix[j][i]+matrix[i][k]);
                  }
               }
            }
```

```
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
              if( matrix[i][j]==1e9)matrix[i][j]=-1;
            }
        }
    }
```

8.KRUSKAL

```
int parent[1200];
int siz[1200];
int Rank[1200];
void make(int x){
  parent[x]=x;
  siz[x]=1;
}
int Find (int x){
 if (parent[x]==x)return x;
 return parent[x]=Find(parent[x]);
}
void UnionBySize(int a,int b){
   a=Find(a);
   b=Find(b);
   if(a!=b){
      if(siz[a]<siz[b]){
         swap(a,b);
      }
      parent[b]=a;
      siz[a]+=siz[b];
   }
}
void UnionByRank(int a,int b){
   a=Find(a);
   b=Find(b);
   if(a==b)return;
   if(Rank[a]<Rank[b]){
      parent[a]=b;
   }
   else if(Rank[a]>Rank[b]){
     parent[b]=a;
   }
   else{
      parent[a]=b;
      Rank[b]++;
   }
}
   int spanningTree(int V, vector<vector<int>>
adj[])
   {
      // code here
      for(int i=0;i<V;i++){
        parent[i]=i;
        Rank[i]=0;
      }
```

```
    vector<pair<int,pair<int,int>>>a;
    for(int i=0;i<V;i++){
      for(auto u:adj[i]){
      a.push_back({u[1],{u[0],i}});
      }
    }
    int ans=0;
    sort(a.begin(),a.end());
    for(auto u:a){
if(Find(u.second.first)!=Find(u.second.second)){
        ans+=u.first;
UnionByRank(u.second.first,u.second.second);
      }
    }
    return ans;
  }
```

9.SCC

```
stack<int>st;
        void dfs(int i,vector<vector<int>>&
adj,vector<int>&vis){
            vis[i]=1;
            for(auto u:adj[i]){
              if(vis[u]==0)dfs(u,adj,vis);
            }
            st.push(i);
           // cout<<i<<endl;
          }
          void dfs1(int
i,vector<int>a[],vector<int>&vis){
            vis[i]=1;
            for(auto u:a[i]){
              if(!vis[u])dfs1(u,a,vis);
            }
          }
          //Function to find number of strongly
connected components in the graph.
  int kosaraju(int V, vector<vector<int>>& adj)
  {
    //code here
    //sort by ending time
    vector<int>vis(V,0);
   //  stack<int>st;
    for(int i=0;i<V;i++){
      if(vis[i]==0){
        dfs(i,adj,vis);
      }
    }
    //reverse edge
    vector<int>a[V];
    for(int i=0;i<V;i++){
      vis[i]=0;
      for(auto u:adj[i]){
        a[u].push_back(i);
```

```
    }
  }
  ///scc component
//   for(auto u:st)cout<<u<<" ";
  int ans=0;
  while(!st.empty()){
    int i=st.top();
   // cout<<i<<" ";
    st.pop();
    if(!vis[i]){
      ans++;
      dfs1(i,a,vis);
    }
  }
  return ans;
}
```

11. detect cycle in a directed graph

```
bool dfs(int i,vector<int>adj[],bool vis[],bool path[]){
    vis[i]=1;
    path[i]=1;
    for(auto u:adj[i]){
      if(vis[u]==0){
      if(dfs(u,adj,vis,path)==true)return true;
    }
    else if(path[u]==1)return true;
  }
  path[i]=0;
  return false;
}
```

12. detect cycle in undirected graph

```
bool isCycle(int V, vector<int> adj[]) {
    // Code here
    int vis[V]={0};
    queue<pair<int,int>>q;
    for(int i=0;i<V;i++){
      if(vis[i]==0){
        q.push({i,-1});
        vis[i]=1;
    while(!q.empty()){
      int x=q.front().first;
      int y=q.front().second;
      q.pop();
      for(auto u:adj[x]){
        if(vis[u]==0){
          vis[u]=1;
          q.push({u,x});
        }
        else{
```

```
                if(u!=y)return true;
              }
            }
          }
        }
          }
        }
        return false;
    }
```

DS

1.segment tree

```
#include<bits/stdc++.h>
using namespace std;
#define ll   long long
#define pb   push_back
#define yes   cout<<"YES"<<endl
#define no   cout<<"NO"<<endl
#define all(x) (x).begin(), (x).end()
#define    gcd(a,b)     __gcd(a,b)
#define    lcm(a,b)  (a*b)/__gcd(a,b)
#define optimize()
ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(
0);
const ll mx=2e5+123;
ll t[mx*4] , a[mx];
void ini ( ll id , ll start , ll end )
{
   if(start==end) {
      t[id]=a[start];
return;
}
   ll mid=(start+end)>>1;
   ini ( id*2 , start , mid );
   ini ( id*2+1 , mid+1, end);
    t[id]=t[id*2]+t[id*2+1];
}
void upd ( ll id, ll b, ll e, ll i, ll val )
{
   if ( b > i || e < i ) return;
   if ( b == e && b == i ) {
      t[id] = val;
      return;
   }
   ll mid = ( b + e ) >> 1;
   upd ( id*2, b, mid, i, val );
   upd ( id*2+1, mid+1, e, i, val );
   t[id] = t[id*2] + t[id*2+1];
}
ll ask( ll id, ll start, ll end, ll l, ll r)
{
   if(start>r || end<l)return 0;
                  if( l<=start && end<=r)
                  {
                     return t[id];
                  }
                  ll mid=(start+end)>>1;
                  ll leftsum=ask( id*2, start, mid, l, r);
                  ll rightsum=ask( id*2+1, mid+1, end, l, r );
                  return leftsum+rightsum;
               }
```

```
void solve(){
   ll n,q;
   cin>>n>>q;
   for(ll i=1; i<=n; i++)cin>>a[i];
   ini(1,1,n);
   while(q--){
    ll typ;
      cin >> typ;
      if ( typ == 1 ) {
        ll k, u;
         cin >> k >> u;
         upd ( 1, 1, n, k, u ); /// O(log2(n))
      }
      else {
         ll l, r;
        cin >> l >> r;
        cout << ask ( 1, 1, n, l, r ) << endl; ///
O(log2(n))
     }
  }
}
```

2.seg tree (lazy)

```
const ll mx=2e5+123;
bool vis[mx*3];
ll prop[mx*3];
ll a[mx];
void shift( ll id )
{
   prop[id*2]+=prop[id];
   prop[id*2+1]+=prop[id];
   vis[id*2]=vis[id*2+1]=1;
   prop[id]=vis[id]=0;
}
void init( ll id, ll b, ll e)
{
   if(b==e){
                  prop[id]=a[b];
                  return ;
               }
               ll mid=(b+e)>>1;
               init ( id*2, b, mid);
               init( id*2+1 , mid+1, e);
            }
            void upd ( ll id, ll b, ll e, ll i, ll j, ll val )
            {
               if ( b > j || e < i ) return;
               if ( b >= i && e <= j ) {
                  prop[id] += val;
                  vis[id] = 1;
                  return;
               }
               ll mid = ( b + e ) >> 1;
               upd ( id*2, b, mid, i, j, val );
               upd ( id*2+1, mid+1, e, i, j, val );
            }
```

```
        ll ask ( ll id, ll b, ll e, ll i )
        {
            if ( i < b || i > e ) return 0;
            if ( b == e && i == b ) return prop[id];
    if ( vis[id] ) shift ( id );
    ll mid = ( b + e ) >> 1;
    ll left = ask ( id*2, b, mid, i );
    ll right = ask ( id*2+1, mid+1, e, i );
    return left + right;
}
void solve(){
    ll n,q ;
    cin>>n>>q;
    for(ll i=1; i<=n; i++)cin>>a[i];
    init( 1, 1, n);
    while(q--)
    {
        ll type;
        cin>>type;
        if(type==1){
            ll a, u, b;
            cin>>a>>b>>u;
            upd( 1, 1, n, a, b, u);
        }
        else{
            ll k;
            cin>>k;
            cout<<ask( 1, 1, n, k)<<endl;
        }
    }
}
```

3.Histogram

```
void answer()
{
    ll n;
    cin>>n;
    vector<ll>v(n);
    for(int i=0;i<n;i++)cin>>v[i];
    vector<ll>p(n,0);
    vector<ll>s(n,0);
    stack<pii>st;
    for(int i=0;i<n;i++){
        ll c=1;
       // cout<<v[i]<<endl;
        while(!st.empty()&&st.top().F>=v[i]){
            //cout<<st.top()<<" ";
          //  cout<<p[st.top().S]<<endl;
            c+=p[st.top().S];
            st.pop();
        }
        st.push({v[i],i});
        p[i]=c;
    }
    while(!st.empty())st.pop();
```

```
    for(int i=n-1;i>=0;i--){
      ll c=1;
      while(!st.empty()&&st.top().F>=v[i]){
        c+=s[st.top().S];
         st.pop();
      }
   st.push({v[i],i});
      s[i]=c;
    }
    ll ans=0;
    for(int i=0;i<n;i++){
        //cout<<p[i]<<" "<<s[i]<<endl;
      ll c=p[i]+s[i]-1;
      ans=max(ans,v[i]*c);
    }
    cout<<ans<<endl;
}
```

4.MOS

```
ll b;
ll cnt[mx];
bool cmpr(pii x,pii y){
  if(x.F/b==y.F/b)return x.S<y.S;
  return x.F/b<y.F/b;
}
void answer()
{
ll n;
cin>>n;
vector<ll>v(n);
for(int i=0;i<n;i++)cin>>v[i];
vector<pii>query;
vector<pii>a;
int q;
cin>>q;
b=sqrt(n);
while(q--){
   ll x,y;
   cin>>x>>y;
   x--;
   y--;
   query.pb({x,y});
   a.pb({x,y});
}
a.pb({n,n});
sort(a.begin(),a.end(),cmpr);
map<pii,ll>mp;
ll c=0;
memset(cnt,0,sizeof(cnt));
ll lp=a[0].F,rp=a[0].S;
for(int i=lp;i<=rp;i++){
  if(cnt[v[i]]==0)c++;
  cnt[v[i]]++;
}
mp[{lp,rp}]=c;
for(int i=1;i<a.size()-1;i++){
```

```
ll x=a[i].F;
ll y=a[i].S;
while(lp!=x){
  if(x>lp){
     ll val=v[lp];
     cnt[val]--;
     if(cnt[val]==0)c--;
     lp++;
  }
  else {
     lp--;
     ll val=v[lp];
     if(cnt[val]==0)c++;
     cnt[val]++;
  }
}
while(rp!=y){
  if(y>rp){
     rp++;
     ll val=v[rp];
     if(cnt[val]==0)c++;
     cnt[val]++;
  }
  else {
     ll val=v[rp];
     rp--;
      cnt[val]--;
      if(cnt[val]==0)c--;
  }
      }
      mp[{x,y}]=c;
  }
  for(auto u:query){
     cout<<mp[{u.F,u.S}]<<endl;
  }
}
```

5.PBDS

```
#include<ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
typedef tree<long long, null_type, less<long long>,
rb_tree_tag,tree_order_statistics_node_update
>ordered_set;//for set
```

//less er jaigay greater dile boro theke choto set

```
// Multiset: use pair<value, unique_id>
// comparator: less<pair<ll,int>>
// Never use less_equal in PBDS tree
// (strict ordering is required)
```

//less er jaigay greater dile boro theke choto multiset

///cout<<x(name of set or multiset).order_of_key(-5(parameter))<<endl;//number of items in a set that are stricly smaller than

///our item

///cout<<x(name of set or multiset).find_by_order(1(parameter))<<endl;//find the item in index 1

//for erase

```
int x=*st.find_by_order(j);
st.erase(st.find_by_order(st.order_of_key(x)));
```

STRING

1.KMP

```
void answer()
{
   string text;
   cin>>text;
   string pattern;
   cin>>pattern;
  // cout<<s<<endl;
   int n=pattern.size();
   vector<int>lcsp(n);
       p[0]=0;
   int j=0;
   for(int i=1;i<n;i++){
      while(j>0&&pattern[j]!=pattern[i]){
         j=lcsp[j-1];
      }
      if(pattern[i]==pattern[j])j++;
      lcsp[i]=j;
   }
 //vector<ll>ans;
 j=0;
  int ans=0;
 for(int i=0;i<text.size();i++){
   if(text[i]==pattern[j])j++;
   else{
     if(j!=0) {
        j=lcsp[j-1];
          i--;
      }
     }
     if(j==n){
          // cout<<i<<endl;
        //ans.pb(((i+1)-n)+1);
         ans++;
        // i--;
         j=lcsp[j-1];
      }
     }
     ///if(ans.empty())ans.pb(-1);
     //for(auto u:ans)cout<<u<<" ";
     cout<<ans<<endl;
    }
```

3.Hashing

```
const ll mod=1000000009,mod2=1000000011;
void answer();
const ll mx=1e6+123;
ll base=31,base2=37;
void answer(){
  vector<ll>pw(mx);
  pw[0]=1;
          for(int i=1;i<mx;i++){
            pw[i]=(pw[i-1]*base)%mod;
          }
```

```
  string s;
  cin>>s;
  ll n=s.size();
  vector<ll>hash(n+1,0);
  ll hashval=0;
  for(int i=0;i<n;i++){
     hashval=(hashval*base+((s[i]-'0')+1))%mod;
     hash[i+1]=hashval;
  }
  for(int i=1;i<n;i++){
    ll a,b,c,d;
    a=hash[i];
    c=hash[n];
    c-=(hash[n-i]*pw[i])%mod;
    if(c<0)c+=mod;
    if(a==c){
      cout<<i<<" ";
    }
  }
}
```

NUMBER THEORY

1.Modular arithematics

```
    const int N=2e6+2;
const ll M=1e9+7;
     long long mod(long long x){
       return (x%M);
     }
     long long add(long long a, long long b){
       return mod(mod(a)+mod(b));
     }
     long long mul(long long a, long long b){
       return mod(mod(a)*mod(b));
     }
     //finding ( a to the power b ) % M;
     ll modPow(ll base, ll power){
       if (!power) return 1; // base case
       ll result = modPow(base, power/2);
         result = (result * result) % M;
       if (power & 1) result = (result * base) %
M;
       return result % M;
     }
     // precalculating factorial
 int fact[N];
     ll inv_fact[N];
     void precalc(){
       fact[0]=1;
       for(int i=1;i<N;i++){
                    fact[i]=mul(fact[i-1],i);
                  }
```

```
        inv_fact[N - 1] = modPow(fact[N - 1], M - 2);
        for (int i = N - 2; i >= 0; i--) {
        inv_fact[i] = mul(inv_fact[i + 1], i + 1);
  }
    }
    ll inv(ll x){
      return modPow(x,M-2);
    }
    ll divide(ll a, ll b){
      return mul(a,inv(b));
    }
    ll nCr(ll n, ll r){
      if(r<0 || r>n || n>=N)return 0;
      if(r==n)return 1;
      return mul(fact[n], mul(inv_fact[r], inv_fact[n - r]));
    }
```

2.Factorization

```
vector<ll> getFactorization(ll x)
{
  vector<ll> ret;
  for(auto p:prime){
    if(1LL*p*p>x)break;
    if(x%p==0){
      while(x%p==0){
        ret.push_back(p);
        x/=p;
      }
    }
  }
  if(x>1)ret.push_back(x);
  return ret;
}
```

3.Number of divisor

```
ll NOD(ll n)//finding number of devisor
{
  ll ret=1;
  for(auto p: prime) //using seive for making prime
    if(1LL*p*p>n)break;
    if(n%p==0)
    {
      ll cnt=1;
      while(n%p==0)
      {
        n/=p;
        cnt++;
```

```
        }
        ret*=cnt;
      }
    }
    if(n>1)ret*=2;
    return ret;
}
```

4.Phi

```
ll phi( ll n ){
      ll ret =n;
      for(  auto p: prime) //prime from seave
      {
         if(1LL* p*p>n || n==0)break;
         if(n%p==0)
         {
            ret/=p;
            ret*=(p-1);
            while(n%p==0)n/=p;
         }
      }
      if(n>1)
      {
         ret/=n;
         ret*=(n-1);
      }
      return ret;
}
```

5.Factorization

```
vector<ll> primeFactors(ll n)
{
    vector<ll>PF;
   for(ll i=2; i*i<=n; i++)
   {
      if(n%i==0)
      {
         while(n%i==0)
         {
            PF.push_back(i);
            n/=i;
         }
      }
   }
   if(n>1)PF.push_back(n);
   return PF;
}
```

6.Seve

```
const ll N=1e7+123;
vector<bool>is_prime(N,1);
vector<ll>prime;
void sieve()
{
```

```
    is_prime[0]=false;
    is_prime[1]=false;
    for(ll i=2; i<N; i++)
    {
       if(is_prime[i]==true){
          for(ll j=2*i; j<N; j+=i){
             is_prime[j]=false;
           }
       }
    }
    for(ll i=0; i<N; i++){
       if(is_prime[i])prime.push_back(i);
    }
}
```

7.Sigma function

//This function actually denotes the sum of all divisors of a number. For example σ(24) = 1+2+3+4+6+8+12+24=60.

```
// sigma(n)=Product[(p^(e+1)-1)/(p-1)]
// over prime powers p^e dividing n.
e1 is its occurance in n

// Number of divisors is (k1+1)×⋯×(kn+1)

      // sum of divisor find using sigma function

      // product of divisor = number ^ (number of div)/2;
```

Here k is the power of prime factor

6.Equation

1. Area of trapezium: 0.5 * h * (a+b)
2. Sum of odd number formula: n^2
3. Sum of even number formula: n * (n+1)
4. Sum of squares formula: ⅙*n * (n+1)*(2n+1)
5. Sum of cube series formula: ¼*n^2 * (n+1)^2
6: Series Sum: n/2 * (2a + (n-1)*d)
7. sum(l to r) = (r*(r+1)-(l-1)*l)/2

DP

1.Lis log n

```
vector<int>nums(n);
 for(ll i=0; i<n; i++)cin>>nums[i];

  ll inf=10000000000;
    vector<ll>v(n+1, inf);
    v[0]=-inf;
    for(int i=0; i<n ; i++)
    {
       int j=upper_bound ( v.begin() , v.end() , nums[i])-v.begin();
        if(v[j-1]<nums[i]){
           v[j]=nums[i];
        }
    }
    int ans=0;
    for(int i=0; i<=n; i++){
       if(v[i]!=inf)ans=i;
    }

    cout<<ans<<endl;
```

2.Bitmask

```
const ll mx=(1<<16)+123;
ll dp[20][mx];
ll a[20][20];
ll n;

ll rec ( ll i , ll mask)
{
   if(i==n)return 0;
   if(mask==(1<<n)-1)return 0;
   if(dp[i][mask]!=-1)return dp[i][mask];
   ll ans=0;
   for(ll j=0; j<n; j++){
      if(((1<<j)&mask) == 0){ // chaking
ith bit set or not

         ans=max( ans , a[i][j]+rec(i+1,
(1<<j)| mask));// make ith bit set by or

      }
   }
   return dp[i][mask]=ans;

}

void solve(){

    cin>>n;
    for(ll i=0; i<n; i++)
    {
      for(ll j=0; j<n; j++)cin>>a[i][j];
    }
    memset(dp, -1, sizeof(dp));
    cout<<rec(0,0)<<endl;

}
```

STRESS TESYING

1.Genaretor

```
mt19937_64
rng(chrono::steady_clock::now().time_since_ep
och().count());
inline ll gen_random(ll l, ll r) {
   return uniform_int_distribution<ll>(l, r)(rng);
}
int main(){
      ll waterMelon = gen_random(1,15);
      cout<<waterMelon<<endl;
}
```

2.**Bash_script.sh**

```
for((i = 1; ; ++i)); do
   echo $i
   ./gen $i > input.txt
   diff -w <(./a < input.txt) <(./b < input.txt) ||
break
done
```

//gen=genaretor,a,b= my code and brute force code

Template

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

void solve() {

}

int main() {
   ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);

#ifndef ONLINE_JUDGE
   freopen("input.txt", "r", stdin);
   freopen("output.txt", "w", stdout);
#endif

   int t = 1;
   cin >> t;
   while (t--) {
      solve();
   }
   return 0;
}
```

Miller-Rabin Primality Test (64-bit)

```
using ull = unsigned long long;
using u128 = __uint128_t;

ull mulmod(ull a, ull b, ull m) {
   return (u128)a * b % m;
}
ull powmod(ull a, ull e, ull m) {
   ull r = 1;
   while (e) {
      if (e & 1) r = mulmod(r, a, m);
      a = mulmod(a, a, m);
      e >>= 1;
   }
   return r;
}
bool isPrime(ull n) {
   if (n < 2) return false;
   for (ull p : {2ULL,3ULL,5ULL,7ULL,11ULL,
            13ULL,17ULL,19ULL,23ULL,29ULL,
            31ULL,37ULL}) {
      if (n % p == 0) return n == p;
   }
   ull d = n - 1, s = 0;
   while (!(d & 1)) d >>= 1, ++s;
   for (ull a : {2ULL,325ULL,9375ULL,
            28178ULL,450775ULL,
            9780504ULL,1795265022ULL}) {
      if (a % n == 0) continue;
      ull x = powmod(a % n, d, n);
      if (x == 1 || x == n - 1) continue;
      bool composite = true;
      for (ull r = 1; r < s; ++r) {
         x = mulmod(x, x, n);
         if (x == n - 1) {
            composite = false;
            break;
         }
      }
      if (composite) return false;
   }
   return true;
}
```

Usage: isPrime(n) -> true if prime, else false

Deterministic for unsigned 64-bit; uses __int128 (GNU C++17).

Time: O(log n) modular multiplications (fixed bases).

Queue

```
queue<int> q;

q.push(x);    // Insert element
q.pop();      // Remove front element
q.front();    // Get front element
q.back();     // Get last element
q.size();     // Number of elements
q.empty();    // Check if empty
q.emplace(x); // Insert element
```

Priority Queue

```
priority_queue<int> pq; // Max Heap
priority_queue<int, vector<int>,
            greater<int>> pq2; // Min Heap

pq.push(x);    // Insert element
pq.pop();      // Remove top element
pq.top();      // Get highest priority element
pq.size();     // Number of elements
pq.empty();    // Check if empty
pq.emplace(x); // Insert element
```

Stack

```
stack<int> st;

st.push(x);    // Insert element
st.pop();      // Remove top element
st.top();      // Get top element
st.size();     // Number of elements
st.empty();    // Check if empty
st.emplace(x); // Insert element
```

Note: pop() returns void; check empty() first.

COMBINATORICS & PROBABILITY

1. Circular Permutation

(n-1)!
Use: n distinct objects in a circle;
rotations same, reflections different.

2. Derangement

D(0)=1, D(1)=0
D(n)=(n-1)*(D(n-1)+D(n-2))
Use: No object in original position.

3. Stars and Bars

x1+x2+...+xk=n
xi>=0: (n+k-1)C(k-1)
xi>=1: (n-1)C(k-1)
xi>=L: (n-k*L+k-1)C(k-1)
Use: Distribute identical items
into k distinct groups.
Note: xi>=L needs n>=k*L.

4. At Least One Success

P = 1-(1-p)^n
Use: >=1 success in n independent trials.

5. Exactly k Successes

P = nCk*p^k*(1-p)^(n-k)
Use: Exactly k successes in n trials.

6. At Least k Successes

P = SUM(i=k..n) nCi*p^i*(1-p)^(n-i)
Use: >=k successes in n trials.

7. Expected Value

E(X) = SUM(x*P(X=x))
E(X+Y) = E(X)+E(Y)
E(successes) = n*p
E(tries to 1st success) = 1/p
Use: Expected score, cost, count, moves.

8. Expected Minimum / Maximum

For n IID Uniform(0,1) values:
E(Min) = 1/(n+1)
E(Max) = n/(n+1)

For n IID Uniform integers [1,m]:
E(Min) = SUM(j=1..m) ((m-j+1)/m)^n
E(Max) = SUM(j=1..m) [1-((j-1)/m)^n]

General nonnegative integer IID X:
E(Min) = SUM(t>=1) P(X>=t)^n
E(Max) = SUM(t>=1) [1-P(X<t)^n]
Use: Expected smallest/largest of
n IID random values.

Notation

n = number of objects/trials
k = successes/groups (by context)
p = success probability each trial
nCk = n!/(k!*(n-k)!)
IID = independent, identical dist.
SUM = summation

Contest Notes

Binomial formulas: trials IID,
same success probability p.
Min/Max: choose formula for
the correct distribution.